

sec_25.06.26

Audity — Streamline the Audit Process

Scope: Lokale Produktiv-Installation (Docker-Compose) + Quellcode-Review
Datum: 2026-06-25 · **Version:** Audity 0.2.4 (Branch `production`)
Methodik: Whitebox — Laufzeit-/Container-Inspektion, Konfigurations- & Secret-Review, Dependency-Audit, manuelles Code-Review (Auth, Krypto, Authz, SSRF, Injection, XSS)
Assessor: Security Architect (Claude)

1. Management Summary

Die Anwendung ist **überdurchschnittlich sorgfältig und reif entwickelt**: durchgängiges Threat-Modeling, saubere Autorisierung, gehärtete Container und 0 bekannte Dependency-Schwachstellen. Im **Anwendungscode wurde keine ausnutzbare Schwachstelle** gefunden.

Die relevanten Risiken liegen **auf der Ebene der Deployment-/Betriebs-Posture dieser konkreten Installation** — allen voran der Betrieb der Produktion über **Klartext-HTTP ohne TLS** und ein **world-readable `.env`**, das laut Design-Modell die eigentliche Sicherheitsgrenze ist.



Findings-Übersicht

ID	SCHWERE	TITEL	KATEGORIE
H-1	HOCH	Produktion über Klartext-HTTP (kein TLS)	Transport / Config
M-1	MITTEL	<code>.env</code> world-readable (0644) mit allen Secrets	Secret-Management
M-2	MITTEL	MFA org-weit nicht erzwungen in Produktion	Auth-Policy
M-3	MITTEL	Worker abgestürzt & keine Restart-Policy → Backups stehen	Resilienz / Verfügbarkeit
M-4	MITTEL	Updater-Token nicht von Fail-closed-Prüfung abgedeckt	Secret-Validierung
L-1	NIEDRIG	Seed-Admin-Credential in world-readable <code>.env</code>	Identität
L-2	NIEDRIG	Web-CSP <code>connect-src</code> inkonsistent (localhost-Storage)	Config / CSP
L-3	NIEDRIG	Maintenance-Console in Produktion aktiviert	Angriffsfläche
L-4	NIEDRIG	Updater <code>version</code> -Argument ohne Charset-Restriktion	Hardening
L-5	NIEDRIG	PDF-Renderer (Puppeteer) verlässt sich allein auf <code>escapeHtml</code>	Defense-in-Depth

2. Laufzeit-Bestandsaufnahme (lokale Installation)

CONTAINER	STATUS	HOST-BINDING	BEWERTUNG
audity-web (nginx)	Up (healthy)	127.0.0.1:80	nur Loopback — gut
audity-api	Up (healthy)	127.0.0.1:3000	nur Loopback — gut
audity-updater	Up	kein Host-Port	nur Docker-Netz — gut (Docker-Socket!)
audity-worker	Exited (1)	–	abgestürzt (siehe M-3)
audity-db (postgres)	Up (healthy)	kein Host-Port	nur intern — gut
audity-redis	Up (healthy)	kein Host-Port	nur intern — gut
audity-storage (minio)	Up (healthy)	127.0.0.1:9000-9001	nur Loopback — gut

Positiv: db, redis und der privilegierte Updater sind nicht host-exponiert; alle gemappten Ports binden auf `127.0.0.1`. Der öffentliche Zugang erfolgt offenbar über einen vorgelagerten Layer auf `http://150.230.20.17`.

3. Detaillierte Findings

H-1 **HOCH** Produktion über Klartext-HTTP (kein TLS)

Belege `AUDITY_ENV=production`, `AUDITY_PUBLIC_URL=http://150.230.20.17` (öffentliche IP, kein `https`).

Auswirkung Login-Credentials, das Bearer-Access-Token, der Refresh-Cookie (wird wegen `publicUrl.startsWith("https")` mit `secure=false` gesetzt) sowie Evidence-Downloads werden im Klartext übertragen. Kein HSTS. Ein Netzwerk-/MitM-Angreifer kann Sessions und Zugangsdaten mitlesen und übernehmen.

Ursache `validateProductionConfig()` erzwingt zwar starke Secrets, prüft aber *nicht* auf TLS — HTTP-Produktion passiert die Validierung still.

Empfehlung TLS terminieren (Reverse-Proxy/Caddy/nginx + Let's Encrypt), `AUDITY_PUBLIC_URL` auf `https://` setzen, HSTS aktivieren, HTTP→HTTPS-Redirect. Optional: Start in Produktion bei Nicht-HTTPS- `publicUrl` mit Warnung/Abbruch versehen.

M-1 **MITTEL** `.env` ist world-readable (0644)

Belege `-rw-r--r-- .env` enthält `AUDITY_ENCRYPTION_KEY`, `AUDITY_APP_SECRET`, `AUDITY_POSTGRES_PASSWORD`, `AUDITY_UPDATER_TOKEN`, `AUDITY_SEED_ADMIN_PASSWORD`, Storage-Keys. Zusätzlich wird das Projektverzeichnis (inkl. `.env`) per Bind-Mount in den Updater-Container gehängt.

Auswirkung Jeder lokale Benutzer / jeder andere Prozess auf dem Host kann sämtliche Secrets lesen. Das widerspricht direkt dem dokumentierten Modell, in dem `.env` die eigentliche Sicherheitsgrenze ist.

Empfehlung `chmod 600 .env` (und Eigentümer = Service-User); sicherstellen, dass `install.sh` die Datei mit `umask 077` bzw. `chmod 600` erzeugt.

M-2 **MITTEL** MFA in Produktion nicht erzwungen

Belege `AUDITY_REQUIRE_MFA=false`.

Auswirkung Reguläre Accounts (inkl. Admins) können ohne zweiten Faktor angemeldet werden. In Kombination mit H-1 (Klartext) und L-1 (Seed-Admin) sinkt die Hürde für Account-Takeover deutlich. (Hinweis: die Maintenance-Console erzwingt MFA *pro Admin* unabhängig davon.)

Empfehlung `AUDITY_REQUIRE_MFA=true` für privilegierte Rollen, mindestens für alle Admins.

M-3 **MITTEL** Worker-Crash ohne Auto-Recovery → Backups stehen

Belege `audity-worker` `Exit(1): Error: connect ECONNREFUSED 172.18.0.5:9000` (MinIO kurz nicht erreichbar) → unbehandelte Exception beendet den Prozess. **Kein einziger Service in `docker-compose.prod.yml` definiert eine `restart: -` Policy.**

Auswirkung Nach Crash oder Host-Reboot kommt nichts automatisch zurück. Der Worker verarbeitet Backup- und PDF-Render-Jobs — ein liegender Worker bedeutet, dass die Sicherheitskontrolle „Backup“ still ausfällt.

Empfehlung `restart: unless-stopped` für alle Services; transiente Storage-Fehler im Worker abfangen/retryen statt den Prozess zu beenden; Alerting auf fehlende/fehlgeschlagene Backup-Runs.

M-4 **MITTEL** Updater-Token nicht von Fail-closed-Prüfung erfasst

Belege `validateProductionConfig()` prüft `APP_SECRET`, `ENCRYPTION_KEY`, `DATABASE_URL`, Storage-Keys — aber nicht `AUDITY_UPDATER_TOKEN`. Compose-Default: `replace-with-secure-updater-token`.

Auswirkung Ein unvollständiges Deployment kann den Docker-Socket-Dienst (Host-RCE-Grenze) mit dem bekannten Default-Token betreiben, ohne dass der Start verweigert wird. *In dieser Installation* ist ein starkes 64-Zeichen-Token gesetzt — Risiko ist latent, nicht aktiv.

Empfehlung Updater-Token (und DB-Passwort) in die Fail-closed-Liste aufnehmen; Mindestlänge erzwingen.

L-1 **NIEDRIG** Seed-Admin-Credential in `.env`

Detail `db/seed.ts` legt den initialen Instance-Admin nur bei leerer DB an (kein Re-Injection-Backdoor — gut), doch das Passwort steht dauerhaft in der world-readable `.env` (M-1) und wird nie zur Rotation gezwungen.

Empfehlung Nach Erst-Setup Passwort rotieren und `AUDITY_SEED_ADMIN_PASSWORD` aus `.env` entfernen.

L-2 **NIEDRIG** Web-CSP `connect-src` inkonsistent

Detail Die nginx-CSP des Web-Containers nutzt `connect-src ... http://localhost:9000 http://127.0.0.1:9000` (Default aus `AUDITY_CSP_CONNECT_EXTRA`), während die API auf die öffentliche IP zeigt. Für Remote-Browser ist `localhost:9000` nicht erreichbar.

Empfehlung `AUDITY_CSP_CONNECT_EXTRA / AUDITY_STORAGE_PUBLIC_ENDPOINT` auf die reale (idealerweise `https`) Storage-Origin setzen; keine localhost-Quellen in Produktion.

L-3 **NIEDRIG** Maintenance-Console in Produktion aktiv

Detail `AUDITY_SERVER_CONSOLE_ENABLED=true`. Die Console ist sehr gut abgesichert (Step-up Passwort+MFA, kein Shell, Allowlist, IP-gebundener Grant), erweitert aber die Angriffsfläche.

Empfehlung Nur aktiviert lassen, wenn betrieblich nötig; sonst deaktivieren (Least-Functionality).

L-4 **NIEDRIG** Updater `version` -Argument ohne Charset-Restriktion

Detail `/run` validiert `version` nur als `min(1).max(80)` und reicht es als `$1` an `sh ./scripts/update.sh` weiter. Per `spawn` ist es ein einzelnes argv-Element (kein direktes Shell-Injection), aber wenn `update.sh $1` unquoted in einer Command-Substitution verwendet, ist Injection denkbar (Token vorausgesetzt).

Empfehlung Charset-Allowlist (z. B. nur `[A-Za-z0-9._-]` / SemVer); in `update.sh` konsequent quoten.

L-5 **NIEDRIG** PDF-Renderer verlässt sich allein auf `escapeHtml`

Detail Der Worker rendert Reports via `page.setContent(html_preview)`. Stored-XSS/SSRF ist heute verhindert, weil der HTML server-seitig mit `escapeHtml` auf *jedem* Nutzerfeld gebaut und vor jedem Render neu erzeugt wird (gespeichertes `html_preview` wird bewusst nicht vertraut). Das gesamte Schutzmodell hängt damit an der Vollständigkeit von `escapeHtml`.

Empfehlung Defense-in-Depth: `page.setJavaScriptEnabled(false)` und das Laden von Nicht-`data:`-Ressourcen während des Renderns per Request-Interception blockieren.

4. Geprüft & solide (positive Befunde)

Folgende Bereiche wurden geprüft und zeigen reife, ehrliche Sicherheitsarchitektur — ohne ausnutzbare Schwachstelle:

- **Dependencies:** `npm audit` = 0 Schwachstellen.
- **Secrets:** stark (APP_SECRET 64, ENCRYPTION_KEY 256-bit, DB/Storage/Updater je 64); `.env` nicht in Git; keine committeten Secrets.
- **Container-Härtung:** `read_only` -Rootfs, Non-root-User, gepinnte Image-Digests (Supply-Chain), Loopback-Bindings.
- **Auth:** argon2id, JWT fest HS256 (kein alg-Confusion/ none), purpose-scoped Tokens.
- **MFA:** TOTP + Recovery-Codes; Step-up-Auth bei MFA-Disable & Console; Session-Revoke bei Passwortwechsel.
- **Setup-Race:** Advisory-Lock verhindert doppelten Instance-Admin.
- **Encryption-Key-Seal:** atomares Single-UPDATE, race-frei genau ein Reveal pro Key-Generation.
- **Console:** kein Shell, Allowlist+switch, Docker an token-gated Updater delegiert, Grant an IP+Session gebunden.
- **Updater:** `timingSafeEqual` -Token, `execFile` (kein Shell), Docker-Args aus striktem Enum.
- **Authz:** zentrale `canAccessAssessment/Customer` (Ownership-oder-Share), Evidence IDOR-sicher, traversal-sichere Object-Keys.
- **CSRF:** Double-Submit-Token im Custom-Header + Origin-Host-Check; `trustProxy` auf private Peers begrenzt (kein IP-Spoofing).
- **Public-API & Portal:** gehashte Tokens, scoped, rate-limited, parametrisiert; eingefrorene Snapshots.
- **Fail-closed:** `validateProductionConfig` verweigert Default-/schwache/duplizierte Kern-Secrets.

5. Remediation-Priorisierung

PRIORITÄT	MASSNAHME	FINDINGS	AUFWAND
1 (sofort)	TLS aktivieren + <code>AUDITY_PUBLIC_URL=https://...</code> + HSTS	H-1	mittel
1 (sofort)	<code>chmod 600 .env</code> ; Seed-Admin-Passwort rotieren & aus <code>.env</code> entfernen	M-1, L-1	gering
2 (kurzfristig)	<code>restart: unless-stopped</code> + Worker-Fehler abfangen + Backup-Alerting; Worker neu starten	M-3	gering
2 (kurzfristig)	MFA für Admins erzwingen	M-2	gering
3 (mittelfristig)	Updater-Token in Fail-closed-Liste; Storage-Public-Endpoint/CSP korrigieren; Console nur bei Bedarf	M-4, L-2, L-3	gering
3 (Härtung)	Updater- <code>version</code> -Charset; Puppeteer JS-disable + Resource-Block	L-4, L-5	gering

Hinweis zur Methodik & Grenzen. Whitebox-Assessment mit Laufzeit-Inspektion, Konfig-/Secret-Review, Dependency-Audit und manuellem Code-Review der sicherheitskritischen Pfade (Auth, Krypto, Authz, Console/Updater, Log-Archive, Public-API, CustomerAck-Portal, Evidence/Storage, Reports-Rendering, CSRF/Origin). Es wurde *kein* aktiver Exploit gegen die Live-Instanz ausgeführt und keine Secret-Werte ausgelesen/abgebildet. Nicht im Detail geprüft: Connectors-/LLM-Integrationen, SMTP-Pfad, sämtliche Frontend-Komponenten. Bewertungen sind risikobasiert; CVSS-Scores wurden nicht formal berechnet. — Erstellt 2026-06-25.